

Normalization in Redux / Redux Toolkit:

Normalization in Redux means restructuring the data in your state so that:

- Each item (like a user, post, comment, etc.) is stored only once.
- Instead of nesting or duplicating full objects, items are referenced by their ID.

This approach makes the Redux state:

- Flat (not deeply nested)
- Easier to update
- Less memory-hungry
- More predictable

Why Normalization Is Needed

Problem Without Normalization:

If a piece of data (like a user object) is used in multiple places, and you store the whole object each time, you:

- Waste memory by repeating the same content.
- Must update the same data in multiple locations, increasing chances of bugs.
- Make selectors and reducers more complex because deep nesting is important.

Solution With Normalization:

- You store that user once, in a central place (like a user's slice).

- Other entities (like posts, comments) just store the user's ID, not the full object.
- To get full user data, you look up the user by ID from the central place.

What Does a Normalized State Look Like?

A normalized Redux state usually uses this structure:

```
{
  entities: {
    users: {
      byId: {
        u1: { id: 'u1', name: 'Alice' },
        u2: { id: 'u2', name: 'Bob' }
      },
      allIds: ['u1', 'u2']
    },
    posts: {
      byId: {
        p1: { id: 'p1', title: 'Hello', authorId: 'u1' },
        p2: { id: 'p2', title: 'World', authorId: 'u2' }
      },
      allIds: ['p1', 'p2']
    }
  }
}
```

This format stores:

- byId: for quick lookup using IDs.

- allIds: to maintain ordering and iteration.

Memory Efficiency: How Normalization Helps:

Without Normalization:

- Every post might include the full user object.
- If a user has 100 posts, their data is repeated 100 times.
- This leads to high memory usage and slower updates.

With Normalization:

- Each user is stored once.
- Posts only store authorId.
- You avoid duplicate data by saving memory.

In numbers:

- Without normalization: 100 users × 100 posts = up to **10,000 objects**
- With normalization: 100 users + 100 posts = **200 objects**

So, normalization reduces memory footprint significantly.

Redux Toolkit and Normalization

Redux Toolkit provides a helper called createEntityAdapter to make normalization easy:

1. It automatically builds the byId and allIds structure.
2. It gives you methods to:
 - Add/update/remove entities
 - Access entities by ID or get all entities in order

This removes the need to manually write logic for managing normalized data.

When to Use Normalization

Use normalization when:

- You have related entities (e.g., users and posts, or comments and posts).
- One entity is used multiple times in the state.
- You want to keep your state clean, flat, and memory friendly.
- You work with large collections of data.